

S A D D

PATTERN PROBLEMS

29 Patterns — Complete Playbook

Each pattern: Output + Explanation + Pseudocode + C++ + Java + Python

C++ · Java · Python



👤 DSA DOJO — BATCH 3

by Sans Aura · @sansaura_

PATTERN 01 · Right-Angled Triangle (Numbers)

OUTPUT (n=5)

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

💡 KEY INSIGHT

Outer loop i from $1 \rightarrow n$ (rows)
Inner loop j from $1 \rightarrow i$ (print j)
Each row i has exactly i numbers

PSEUDOCODE

```
for i = 1 to n:
    for j = 1 to i:
        print(j, end=" ")
    print() // newline
```

PATTERN 01 CODE · Right-Angled Triangle (Numbers)

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        cout<<j<<" ";  
    cout<<"\n";  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        System.out.print(j+" ");  
    System.out.println();  
}
```

Python

```
for i in range(1,n+1):  
    for j in range(1,i+1):  
        print(j,end=" ")  
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 02 · Right-Angled Triangle (Stars)

OUTPUT (n=5)

```
*  
* *  
* * *  
* * * *  
* * * * *
```

💡 KEY INSIGHT

Outer loop i = rows 1→n
Inner loop j = print i stars
Simplest pattern — master this first

PSEUDOCODE

```
for i = 1 to n:  
    for j = 1 to i:  
        print("*", end=" ")  
    print() // newline
```

PATTERN 02 CODE · Right-Angled Triangle (Stars)

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        cout<<"* ";  
    cout<<"\n";  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        System.out.print("* ");  
    System.out.println();  
}
```

Python

```
for i in range(1,n+1):  
    print("* "*i)
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 03 · Pyramid (Stars)

OUTPUT (n=5)

```
  *
 * *
* * *
* * * *
* * * * *
```

KEY INSIGHT

Stars per row $i = 2*i - 1$
Leading spaces per row = $n - i$
Two inner loops: spaces first, then stars

PSEUDOCODE

```
for i = 1 to n:
  for sp = 1 to n-i: print(" ")
  for st = 1 to 2*i-1: print("*")
  print() // newline
```

PATTERN 03 CODE · Pyramid (Stars)

C++

```
for(int i=1;i<=n;i++){
    for(int s=1;s<=n-i;s++) cout<<" ";
    for(int j=1;j<=2*i-1;j++)
        cout<<"*";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int s=1;s<=n-i;s++)
        System.out.print(" ");
    for(int j=1;j<=2*i-1;j++)
        System.out.print("*");
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    print(" "*(n-i)+"*"*(2*i-1))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 04 · Inverted Right Triangle (Stars)

OUTPUT (n=5)

```
* * * * *  
* * * *  
* * *  
* *  
*
```

KEY INSIGHT

Start from n stars, decrease each row
Outer loop i from n→1 (downward)
Inner loop j from 1→i

PSEUDOCODE

```
for i = n downto 1:  
    for j = 1 to i:  
        print("*", end=" ")  
    print() // newline
```


PATTERN 04 CODE · Inverted Right Triangle (Stars)

C++

```
for(int i=n;i>=1;i--){  
    for(int j=1;j<=i;j++)  
        cout<<"* ";  
    cout<<"\n";  
}
```

Java

```
for(int i=n;i>=1;i--){  
    for(int j=1;j<=i;j++)  
        System.out.print("* ");  
    System.out.println();  
}
```

Python

```
for i in range(n,0,-1):  
    print("* "*i)
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 05 · Inverted Pyramid (Stars)

OUTPUT (n=5)

```
* * * * *
 * * * *
  * * *
   * *
    *
```

💡 KEY INSIGHT

Spaces grow, stars shrink each row
Row i: $(i-1)$ spaces + $(2*(n-i)+1)$ stars
Mirror of the pyramid pattern

PSEUDOCODE

```
for i = 1 to n:
    for sp = 1 to i-1: print(" ")
    for st = 1 to 2*(n-i)+1: print("*")
    print() // newline
```

PATTERN 05 CODE · Inverted Pyramid (Stars)

C++

```
for(int i=1;i<=n;i++){
    for(int s=1;s<i;s++) cout<<" ";
    for(int j=1;j<=2*(n-i)+1;j++)
        cout<<"*";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int s=1;s<i;s++)
        System.out.print(" ");
    for(int j=1;j<=2*(n-i)+1;j++)
        System.out.print("*");
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    print(" "*(i-1)+"*"*(2*(n-i)+1))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 06 · Half Diamond (Stars)

OUTPUT (n=5)

```
*  
* *  
* * *  
* * * *  
* * * * *  
* * * *  
* * *  
* *  
*
```

KEY INSIGHT

Upper half: stars increase $1 \rightarrow n$
Lower half: stars decrease $n-1 \rightarrow 1$
Two separate loops OR use abs trick

PSEUDOCODE

```
// Upper half  
for i = 1 to n: print("* " * i)  
  
// Lower half  
for i = n-1 downto 1:  
    print("* " * i)
```

PATTERN 06 CODE · Half Diamond (Stars)

C++

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++) cout<<"* ";
    cout<<"\n";
}
for(int i=n-1;i>=1;i--){
    for(int j=1;j<=i;j++) cout<<"* ";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++)
        System.out.print("* ");
    System.out.println();
}
for(int i=n-1;i>=1;i--){
    for(int j=1;j<=i;j++)
        System.out.print("* ");
    System.out.println();
}
```

Python

```
for i in range(1,n+1): print("* "*i)
for i in range(n-1,0,-1): print("* "*i)
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 07 · Full Diamond (Stars, centered)

OUTPUT (n=5)

```
  *
 * *
* * *
* * * *
* * * * *
  * * * *
    * * *
      * *
        *
```

💡 KEY INSIGHT

Upper half: pyramid (spaces= $n-i$, stars= $2*i-1$)
Lower half: inverted pyramid (i from $n-1 \rightarrow 1$)
Same formula both halves — just loop direction changes

PSEUDOCODE

```
// Upper pyramid
for i = 1 to n:
    print(" "*(n-i) + "*"*(2*i-1))

// Lower inverted pyramid
for i = n-1 downto 1:
    print(" "*(n-i) + "*"*(2*i-1))
```

PATTERN 07 CODE · Full Diamond (Stars, centered)

C++

```
for(int i=1;i<=n;i++){
    for(int s=1;s<=n-i;s++) cout<<" ";
    for(int j=1;j<=2*i-1;j++)
        cout<<"*";
    cout<<"\n";
}
for(int i=n-1;i>=1;i--){
    for(int s=1;s<=n-i;s++) cout<<" ";
    for(int j=1;j<=2*i-1;j++)
        cout<<"*";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int s=1;s<=n-i;s++)
        System.out.print(" ");
    for(int j=1;j<=2*i-1;j++)
        System.out.print("*");
    System.out.println();
}
for(int i=n-1;i>=1;i--){
    for(int s=1;s<=n-i;s++)
        System.out.print(" ");
    for(int j=1;j<=2*i-1;j++)
        System.out.print("*");
    System.out.println();
}
```

Python

```
for i in range(1,n+1): print(" "*(n-i)+"*"*(2*i-1))
for i in range(n-1,0,-1): print(" "*(n-i)+"*"*(2*i-1))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 08 · Right-Angled Triangle (Alphabets A-B-C)

OUTPUT (n=5)

```
A
A B
A B C
A B C D
A B C D E
```

💡 KEY INSIGHT

Same structure as number triangle
Print `char(A + j - 1)` at column `j`
`A = ASCII 65`; use `chr(65+j-1)` in Python

PSEUDOCODE

```
for i = 1 to n:
    for j = 1 to i:
        print(char(A+j-1), end=" ")
    print() // newline
```


PATTERN 08 CODE · Right-Angled Triangle (Alphabets A-B-C)

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++){  
        cout<<(char) ('A'+j-1)<<" ";  
        cout<<"\n";  
    }  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++){  
        System.out.print((char) ('A'+j-1)+" ");  
        System.out.println();  
    }  
}
```

Python

```
for i in range(1,n+1):  
    for j in range(i):  
        print(chr(65+j),end=" ")  
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 09 · Inverted Triangle (Alphabets)

OUTPUT (n=5)

```
A B C D E
B C D E
C D E
D E
E
```

💡 KEY INSIGHT

Row i starts at letter $A+i$ and goes to $A+n-1$
Outer loop $i \rightarrow n-1$, inner j from $i \rightarrow n-1$
Each row shifts start letter by 1

PSEUDOCODE

```
for i = 0 to n-1:
    for j = i to n-1:
        print(char(A+j), end=" ")
    print() // newline
```

PATTERN 09 CODE · Inverted Triangle (Alphabets)

C++

```
for(int i=0;i<n;i++){  
    for(int j=i;j<n;j++){  
        cout<<(char) ('A'+j)<<" ";  
        cout<<"\n";  
    }  
}
```

Java

```
for(int i=0;i<n;i++){  
    for(int j=i;j<n;j++){  
        System.out.print((char) ('A'+j)+"  
");  
        System.out.println();  
    }  
}
```

Python

```
for i in range(n):  
    for j in range(i,n):  
        print(chr(65+j),end=" ")  
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 10 · Triangle (Same Letter per Row)

OUTPUT (n=5)

```
A
B B
C C C
D D D D
E E E E E
```

💡 KEY INSIGHT

Row i prints letter $(A + i - 1)$ repeated i times
 $ch = A + i - 1$ stays fixed for entire row
Variation of same-number pattern with chars

PSEUDOCODE

```
for i = 1 to n:
    ch = char(A + i - 1)
    for j = 1 to i:
        print(ch, end=" ")
    print() // newline
```

PATTERN 10 CODE · Triangle (Same Letter per Row)

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++){  
        cout<<(char) ('A'+i-1)<<" ";  
        cout<<"\n";  
    }  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++){  
        System.out.print((char) ('A'+i-1)+" ");  
        System.out.println();  
    }  
}
```

Python

```
for i in range(1,n+1):  
    print((chr(64+i)+" ")*i)
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 11 · Decreasing Number Triangle

OUTPUT (n=5)

```
1 0 9 8 7
4 5 6
3 2
1
```

💡 KEY INSIGHT

Numbers count down continuously
Total = $n*(n+1)/2$; print from total → 1
Row i has $n-i+1$ numbers

PSEUDOCODE

```
num = n*(n+1)//2
for i = 1 to n:
    for j = 1 to n-i+1:
        print(num, end=" ")
        num -= 1
    print() // newline
```

PATTERN 11 CODE · Decreasing Number Triangle

C++

```
int num=n*(n+1)/2;
for(int i=1;i<=n;i++){
    for(int j=1;j<=n-i+1;j++)
        cout<<num--<<" ";
    cout<<"\n";
}
```

Java

```
int num=n*(n+1)/2;
for(int i=1;i<=n;i++){
    for(int j=1;j<=n-i+1;j++)
        System.out.print(num--);
    System.out.println();
}
```

Python

```
num=n*(n+1)//2
for i in range(1,n+1):
    for j in range(n-i+1):
        print(num,end=" ")
    num-=1
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 12 · Multiplication Table Grid

OUTPUT (n=5)

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25



KEY INSIGHT

Element at row i , col $j = i * j$
Two loops: outer=rows, inner=cols
Print $i*j$ at each position

PSEUDOCODE

```
for i = 1 to n:  
    for j = 1 to n:  
        print(i*j, end=" ")  
    print() // newline
```


PATTERN 12 CODE · Multiplication Table Grid

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=n;j++){  
        cout<<i*j<<" ";  
        cout<<"\n";  
    }  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=n;j++){  
        System.out.print(i*j+" ");  
        System.out.println();  
    }  
}
```

Python

```
for i in range(1,n+1):  
    for j in range(1,n+1):  
        print(i*j,end=" ")  
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 13 · Pascal's Triangle

OUTPUT (n=5)

```
  1
 1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

KEY INSIGHT

Use nCr formula: c starts at 1, update $c = c \cdot (i-j) / (j+1)$
Leading spaces = $n-i-1$ for centering
One inner loop handles both spaces and values

PSEUDOCODE

```
for i = 0 to n-1:
    print " " * (n-i-1)
    c = 1 // nCr accumulator
    for j = 0 to i:
        print(c, end=" ")
        c = c * (i-j) // (j+1)
    print() // newline
```

PATTERN 13 CODE · Pascal's Triangle

C++

```
for(int i=0;i<n;i++){
    for(int s=0;s<n-i-1;s++) cout<<"
";
    long c=1;
    for(int j=0;j<=i;j++){
        cout<<c<<" ";
        c=c*(i-j)/(j+1);
    }
    cout<<"\n";
}
```

Java

```
for(int i=0;i<n;i++){
    for(int s=0;s<n-i-1;s++)
        System.out.print(" ");
    long c=1;
    for(int j=0;j<=i;j++){
        System.out.print(c+" ");
        c=c*(i-j)/(j+1);
    }
    System.out.println();
}
```

Python

```
for i in range(n):
    print(" "*(n-i-1),end="")
    c=1
    for j in range(i+1):
        print(c,end=" ")
        c=c*(i-j)//(j+1)
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 14 · Floyd's Triangle

OUTPUT (n=5)

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

💡 KEY INSIGHT

Continuous numbering — running counter
Row i has exactly i numbers
Counter starts at 1, increments every print

PSEUDOCODE

```
num = 1
for i = 1 to n:
    for j = 1 to i:
        print(num, end=" ")
        num += 1
    print() // newline
```

PATTERN 14 CODE · Floyd's Triangle

C++

```
int num=1;
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++)
        cout<<num++<<" ";
    cout<<"\n";
}
```

Java

```
int num=1;
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++)
        System.out.print(num+++" ");
    System.out.println();
}
```

Python

```
num=1
for i in range(1,n+1):
    for j in range(i):
        print(num,end=" ")
        num+=1
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 15 · Triangle (Same Number per Row)

OUTPUT (n=5)

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

💡 KEY INSIGHT

Row i prints number i , repeated i times
Inner loop runs exactly i times
Simple — row number = value = count

PSEUDOCODE

```
for i = 1 to n:
    for j = 1 to i:
        print(i, end=" ")
    print() // newline
```

PATTERN 15 CODE · Triangle (Same Number per Row)

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        cout<<i<<" ";  
    cout<<"\n";  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        System.out.print(i+" ");  
    System.out.println();  
}
```

Python

```
for i in range(1,n+1):  
    print((str(i)+" ")*i)
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 16 · 0-1 Triangle (Binary Triangle)

OUTPUT (n=5)

```
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
```

💡 KEY INSIGHT

Element (i,j): print 1 if $(i+j) \% 2 == 0$, else 0
Odd row starts with 1, even row starts with 0
Flip the value after each print

PSEUDOCODE

```
for i = 1 to n:
    start = 1 if i is odd else 0
    for j = 1 to i:
        print(start, end=" ")
        start = 1 - start // flip
    print() // newline
```


PATTERN 16 CODE · 0-1 Triangle (Binary Triangle)

C++

```
for(int i=1;i<=n;i++){
    int st=(i%2==1)?1:0;
    for(int j=1;j<=i;j++){
        cout<<st<<" ";
        st=1-st;
    }
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    int st=(i%2==1)?1:0;
    for(int j=1;j<=i;j++){
        System.out.print(st+" ");
        st=1-st;
    }
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    st=1 if i%2==1 else 0
    for j in range(i):
        print(st,end=" ")
        st=1-st
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 17 · Number Grid with * Separator

OUTPUT (n=5)

```
1*2*3*4
5*6*7*8
9*10*11*12
13*14*15*16
```

KEY INSIGHT

Print numbers $1 \rightarrow n*n$ in row-major order
Separate with * within a row (not after last)
Counter runs continuously across all rows

PSEUDOCODE

```
num = 1
for i = 1 to n:
    for j = 1 to n:
        print(num, end="")
        if j < n: print("*", end="")
        num += 1
    print() // newline
```

PATTERN 17 CODE · Number Grid with * Separator

C++

```
int num=1;
for(int i=1;i<=n;i++){
    for(int j=1;j<=n;j++){
        cout<<num++;
        if(j<n) cout<<"*";
    }
    cout<<"\n";
}
```

Java

```
int num=1;
for(int i=1;i<=n;i++){
    for(int j=1;j<=n;j++){
        System.out.print(num++);
        if(j<n) System.out.print("*");
    }
    System.out.println();
}
```

Python

```
num=1
for i in range(n):
    row=[]
    for j in range(n):
        row.append(str(num))
        num+=1
    print("".join(row))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 18 · Right Triangle (Numbers 1 to i)

OUTPUT (n=5)

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

💡 KEY INSIGHT

Same core as Pattern 1
Each row i prints 1, 2, 3 ... i
Inner j resets to 1 each new row

PSEUDOCODE

```
for i = 1 to n:
    for j = 1 to i:
        print(j, end=" ")
    print() // newline
```

PATTERN 18 CODE · Right Triangle (Numbers 1 to i)

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        cout<<j<<" ";  
    cout<<"\n";  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        System.out.print(j+" ");  
    System.out.println();  
}
```

Python

```
for i in range(1,n+1):  
    print(*range(1,i+1))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 19 · Butterfly Pattern (Stars)

OUTPUT (n=5)

```
*           *
*  *       *  *
*  *  *   *  *  *
*  *  *  *  *  *  *
*  *  *   *  *
*  *       *  *
*           *
```

💡 KEY INSIGHT

Upper half: left stars=i, spaces=2*(n-i), right stars=i
Lower half: mirror of upper (i from n-1→1)
Three inner loops per row: stars, spaces, stars

PSEUDOCODE

```
// Upper half
for i = 1 to n:
    print(" "*i + " "*2*(n-i) + " "*i)

// Lower half
for i = n-1 downto 1:
    print(" "*i + " "*2*(n-i) + " "*i)
```

PATTERN 19 CODE · Butterfly Pattern (Stars)

C++

```
for(int i=1;i<=n;i++){
    for(int j=0;j<i;j++) cout<<"*";
    for(int j=0;j<2*(n-i);j++) cout<<"
";
    for(int j=0;j<i;j++) cout<<"*";
    cout<<"\n";
}
for(int i=n-1;i>=1;i--){
    for(int j=0;j<i;j++) cout<<"*";
    for(int j=0;j<2*(n-i);j++) cout<<"
";
    for(int j=0;j<i;j++) cout<<"*";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int j=0;j<i;j++)
        System.out.print("*");
    for(int j=0;j<2*(n-i);j++)
        System.out.print(" ");
    for(int j=0;j<i;j++)
        System.out.print("*");
    System.out.println();
}
for(int i=n-1;i>=1;i--){
    for(int j=0;j<i;j++)
        System.out.print("*");
    for(int j=0;j<2*(n-i);j++)
        System.out.print(" ");
    for(int j=0;j<i;j++)
        System.out.print("*");
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    print(""*i+" "*(2*(n-i))+""*i)
for i in range(n-1,0,-1):
    print(""*i+" "*(2*(n-i))+""*i)
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 20 · Floyd's Triangle (Alphabets)

OUTPUT (n=5)

```
A
B C
D E F
G H I J
K L M N O
```

💡 KEY INSIGHT

Print alphabets in order like Floyd's triangle
Running char counter from A (ASCII 65)
Row i gets i consecutive letters

PSEUDOCODE

```
ch = 'A'
for i = 1 to n:
    for j = 1 to i:
        print(ch, end=" ")
        ch = next letter
    print() // newline
```


PATTERN 20 CODE · Floyd's Triangle (Alphabets)

C++

```
char ch='A';
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++)
        cout<<ch++<<" ";
    cout<<"\n";
}
```

Java

```
char ch='A';
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++)
        System.out.print(ch+++" ");
    System.out.println();
}
```

Python

```
ch=65
for i in range(1,n+1):
    for j in range(i):
        print(chr(ch),end=" ")
        ch+=1
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 21 · Number Diamond (Up then Down)

OUTPUT (n=5)

```
1
1 2
1 2 3
1 2 3 4
1 2 3
1 2
1
```

💡 KEY INSIGHT

Upper half: rows $1 \rightarrow n$, row i prints $1..i$
Lower half: rows $n-1 \rightarrow 1$, row i prints $1..i$
Two loops — same inner logic, different outer range

PSEUDOCODE

```
// Upper half
for i = 1 to n:
    for j = 1 to i: print(j)
    print()

// Lower half
for i = n-1 downto 1:
    for j = 1 to i: print(j)
    print()
```

PATTERN 21 CODE · Number Diamond (Up then Down)

C++

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++) cout<<j<<"
";
    cout<<"\n";
}
for(int i=n-1;i>=1;i--){
    for(int j=1;j<=i;j++) cout<<j<<"
";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++)
        System.out.print(j+" ");
    System.out.println();
}
for(int i=n-1;i>=1;i--){
    for(int j=1;j<=i;j++)
        System.out.print(j+" ");
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    print(*range(1,i+1))
for i in range(n-1,0,-1):
    print(*range(1,i+1))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 22 · Sandglass / Hourglass (Stars)

OUTPUT (n=5)

```
* * * * *
 * * *
  *
   *
 * * *
* * * * *
```

💡 KEY INSIGHT

Upper: spaces grow $0 \rightarrow n-1$, stars shrink $n \rightarrow 1$
Lower: spaces shrink $n-2 \rightarrow 0$, stars grow $2 \rightarrow n$
Mirror the upper half for the lower

PSEUDOCODE

```
// Upper (shrinking)
for i = 0 to n-1:
    print(" " * i + "*" * (n-i))

// Lower (growing)
for i = n-2 downto 0:
    print(" " * i + "*" * (n-i))
```

PATTERN 22 CODE · Sandglass / Hourglass (Stars)

C++

```
for(int i=0;i<n;i++){
    for(int s=0;s<i;s++) cout<<" ";
    for(int j=0;j<n-i;j++) cout<<"* ";
    cout<<"\n";
}
for(int i=n-2;i>=0;i--){
    for(int s=0;s<i;s++) cout<<" ";
    for(int j=0;j<n-i;j++) cout<<"* ";
    cout<<"\n";
}
```

Java

```
for(int i=0;i<n;i++){
    for(int s=0;s<i;s++)
        System.out.print(" ");
    for(int j=0;j<n-i;j++)
        System.out.print("* ");
    System.out.println();
}
for(int i=n-2;i>=0;i--){
    for(int s=0;s<i;s++)
        System.out.print(" ");
    for(int j=0;j<n-i;j++)
        System.out.print("* ");
    System.out.println();
}
```

Python

```
for i in range(n): print(" "*i+"*"+
"*(n-i))
for i in range(n-2,-1,-1): print("
"*i+"* "+
"*(n-i))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 23 · Square Border (Stars)

OUTPUT (n=5)

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

KEY INSIGHT

Print * if on border row ($i==0$ or $i==n-1$)
OR on border column ($j==0$ or $j==n-1$)
Else print space — one condition check

PSEUDOCODE

```
for i = 0 to n-1:
    for j = 0 to n-1:
        if border: print("*")
        else: print(" ")
    print() // newline

border = (i==0 or i==n-1
         or j==0 or j==n-1)
```

PATTERN 23 CODE · Square Border (Stars)

C++

```
for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
        if(i==0||i==n-1||j==0||j==n-1)
            cout<<"*";
        else cout<<" ";
    }
    cout<<"\n";
}
```

Java

```
for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
        if(i==0||i==n-1||j==0||j==n-1)
            System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
```

Python

```
for i in range(n):
    for j in range(n):
        if i==0 or i==n-1 or j==0 or
j==n-1:
            print("*",end="")
        else:
            print(" ",end="")
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 24 · Number Diamond Border (min formula)

OUTPUT (n=5)

```
4 4 4 4 4 4 4
4 3 3 3 3 3 4
4 3 2 2 2 3 4
4 3 2 1 2 3 4
4 3 2 2 2 3 4
4 3 3 3 3 3 4
4 4 4 4 4 4 4
```

💡 KEY INSIGHT

Value at (i,j) = $n - \min(i, j, \text{size}-1-i, \text{size}-1-j)$
Distance from nearest border = layer number
 $\text{size} = 2*n - 1$; no nested ifs needed

PSEUDOCODE

```
size = 2*n - 1
for i = 0 to size-1:
    for j = 0 to size-1:
        val = n - min(i, j,
                       size-1-i, size-1-j)
        print(val, end=" ")
    print() // newline
```


PATTERN 24 CODE · Number Diamond Border (min formula)

C++

```
int size=2*n-1;
for(int i=0;i<size;i++){
    for(int j=0;j<size;j++){
        int v=n-min({i,j,size-1-i,size-1-j});
        cout<<v<<" ";
    }
    cout<<"\n";
}
```

Java

```
int size=2*n-1;
for(int i=0;i<size;i++){
    for(int j=0;j<size;j++){
        int v=n-Math.min(Math.min(i,j),
            Math.min(size-1-i,size-1-j));
        System.out.print(v+" ");
    }
    System.out.println();
}
```

Python

```
size=2*n-1
for i in range(size):
    for j in range(size):
        print(n-min(i,j,size-1-i,size-1-j),end=" ")
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 25 · Star + Hash Mixed Triangle

OUTPUT (n=5)

```
*  
* #  
* # *  
* # * #  
* # * # *
```

💡 KEY INSIGHT

Row i has i elements alternating $*$ and $\#$
Odd position (1,3,5...) $\rightarrow *$, even $\rightarrow \#$
Flip symbol after each print in inner loop

PSEUDOCODE

```
for i = 1 to n:  
    for j = 1 to i:  
        if j is odd: print("* ")  
        else: print("# ")  
    print() // newline
```

PATTERN 25 CODE · Star + Hash Mixed Triangle

C++

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++){
        cout<<(j%2==1?"* ":"# ");
        cout<<"\n";
    }
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++){
        System.out.print(j%2==1?"* ":"# ");
    }
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    for j in range(1,i+1):
        print("*" if j%2==1 else
"#",end=" ")
    print()
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 26 · Full Diamond (Alphabets)

OUTPUT (n=5)

```
  A
 A B
A B C
A B C D
A B C D E
  A B C D
   A B C
    A B
     A
```



KEY INSIGHT

Upper half: pyramid with A..i letters per row
Lower half: inverted pyramid
Spaces = n-i for upper; n-i for lower too

PSEUDOCODE

```
// Upper half
for i = 1 to n:
    print(" "*(n-i) + "A B C..."[:i])

// Lower half
for i = n-1 downto 1:
    print(" "*(n-i) + "A B C..."[:i])
```

PATTERN 26 CODE · Full Diamond (Alphabets)

C++

```
for(int i=1;i<=n;i++){
    for(int s=1;s<=n-i;s++) cout<<" ";
    for(int j=0;j<i;j++)
        cout<<(char) ('A'+j)<<" ";
    cout<<"\n";
}
for(int i=n-1;i>=1;i--){
    for(int s=1;s<=n-i;s++) cout<<" ";
    for(int j=0;j<i;j++)
        cout<<(char) ('A'+j)<<" ";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int s=1;s<=n-i;s++)
        System.out.print(" ");
    for(int j=0;j<i;j++)
        System.out.print((char) ('A'+j)+" ");
    System.out.println();
}
for(int i=n-1;i>=1;i--){
    for(int s=1;s<=n-i;s++)
        System.out.print(" ");
    for(int j=0;j<i;j++)
        System.out.print((char) ('A'+j)+" ");
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    print(" "*(n-i)+
          ".join([chr(65+j) for j in
                  range(i)])")
for i in range(n-1,0,-1):
    print(" "*(n-i)+
          ".join([chr(65+j) for j in
                  range(i)])")
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 27 · Reverse Alphabet Triangle (Diamond)

OUTPUT (n=5)

```
A B C D E F G
 A B C D E F
  A B C D E
   A B C D
    A B C
     A B
      A
```

💡 KEY INSIGHT

Upper half only (or extend for full diamond)
Row i: print i leading spaces then A...(n-i) letters
Spaces grow 0→n-1, letters shrink n→1

PSEUDOCODE

```
for i = 0 to n-1:
    for sp = 0 to i-1: print(" ")
    for j = 0 to n-i-1:
        print(char(A+j), end=" ")
    print() // newline
```

PATTERN 27 CODE · Reverse Alphabet Triangle (Diamond)

C++

```
for(int i=0;i<n;i++){  
    for(int s=0;s<i;s++) cout<<" ";  
    for(int j=0;j<n-i;j++)  
        cout<<(char) ('A'+j)<<" ";  
    cout<<"\n";  
}
```

Java

```
for(int i=0;i<n;i++){  
    for(int s=0;s<i;s++)  
        System.out.print(" ");  
    for(int j=0;j<n-i;j++)  
        System.out.print((char) ('A'+j)+"");  
    System.out.println();  
}
```

Python

```
for i in range(n):  
    print(" "*i+" ".join([chr(65+j)  
        for j in range(n-i)]))
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 28 · Right Triangle (Stars, spaced)

OUTPUT (n=5)

```
*  
* *  
* * *  
* * * *  
* * * * *
```

💡 KEY INSIGHT

Classic right triangle — each row i has i stars
Stars separated by spaces
Simplest pattern variation

PSEUDOCODE

```
for i = 1 to n:  
    for j = 1 to i:  
        print("*", end=" ")  
    print() // newline
```


PATTERN 28 CODE · Right Triangle (Stars, spaced)

C++

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        cout<<"* ";  
    cout<<"\n";  
}
```

Java

```
for(int i=1;i<=n;i++){  
    for(int j=1;j<=i;j++)  
        System.out.print("* ");  
    System.out.println();  
}
```

Python

```
for i in range(1,n+1):  
    print("* "*i)
```

💡 Input: n = size of the pattern. All three languages produce the same output.

PATTERN 29 · Number Diamond (Full, Up-Down)

OUTPUT (n=5)

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

💡 KEY INSIGHT

Upper half rows $1 \rightarrow n$, lower half $n-1 \rightarrow 1$
Row i prints 1 to i each time
Full diamond shape using numbers

PSEUDOCODE

```
// Upper half
for i = 1 to n:
    for j = 1 to i: print(j)
    print()

// Lower half
for i = n-1 downto 1:
    for j = 1 to i: print(j)
    print()
```

PATTERN 29 CODE · Number Diamond (Full, Up-Down)

C++

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++) cout<<j<<"
";
    cout<<"\n";
}
for(int i=n-1;i>=1;i--){
    for(int j=1;j<=i;j++) cout<<j<<"
";
    cout<<"\n";
}
```

Java

```
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++)
        System.out.print(j+" ");
    System.out.println();
}
for(int i=n-1;i>=1;i--){
    for(int j=1;j<=i;j++)
        System.out.print(j+" ");
    System.out.println();
}
```

Python

```
for i in range(1,n+1):
    print(*range(1,i+1))
for i in range(n-1,0,-1):
    print(*range(1,i+1))
```

💡 Input: n = size of the pattern. All three languages produce the same output.



DSA DOJO BATCH 3

@sansaura_

instagram · youtube
sansaura.in

ALL 29 PATTERNS

Done. Now write them from memory. 🔥

-
- ✓ **Look at output only — write code from scratch**
 - ✓ Identify: how many loops? spaces? stars? chars?
 - ✓ Know the 3 key formulas: rows, cols, space count
 - ✓ Post your solution in the Dojo group